

Linear classifiers

Introduction

- Up until now, our classifiers have been designed based on probability density functions.
- In some cases, these classifiers were equivalent to linear discriminant functions.
- Now, we will design linear discriminant functions, *regardless of the underlying distribution*.
- Linear classifiers are very useful:
 - "First thing you try".
 - Often an underlying part of more complex algorithms.

A high level view of linear classifiers

- Assume we have a training set: $\{\mathbf{x}_i, y_i\}$, where $\mathbf{x} \in \mathbb{R}^d$
- In terms of discriminant functions:
 - $\mathbf{x}$ belongs to class 1 if $g(\mathbf{x}) > 0$.
 - How to design g ?
- Can use:

- $g(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$.

- $\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \\ w_0 \end{bmatrix}$ and $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \\ 1 \end{bmatrix}$

Mean squared error (MSE) classifier

- Idea:
 - $g(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i \approx y_i$
- How can we find weights that gives the desired output?
- We do this by designing a *cost function* that would encourage this behavior:
 - $J(\mathbf{w}) = \frac{1}{2} \mathbb{E}_{\mathbf{x}} [(y - \mathbf{w}^T \mathbf{x})]$
- $\widehat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} J(\mathbf{w})$
- Want:
 - $\frac{\partial}{\partial \mathbf{w}} J(\mathbf{w}) = 0$

Calculating weights based on MSE loss

- Now, we derive an expression for calculating the weights of a linear classifier with a MSE loss.
- Recall:

$$\blacksquare \mathbb{E}[g(\mathbf{x})] = \int g(\mathbf{x})p(\mathbf{x})d\mathbf{x}$$

- Hence:

$$\blacksquare J(\mathbf{w}) = \frac{1}{2} \int [(y - \mathbf{w}^T \mathbf{x})]^2 p(\mathbf{x})d\mathbf{x}$$

- Using the law of total probability we get:

$$\blacksquare J(\mathbf{w}) = \frac{1}{2}p(w_1) \int [(\text{***} - \mathbf{w}^T \mathbf{x})]^2 p(\mathbf{x}|w_1)d\mathbf{x} + \frac{1}{2}p(w_2)$$

Understanding the MSE weights

$$\int [(\text{***} - \mathbf{w}^T \mathbf{x})]^2 p(\mathbf{x}|w_2)d\mathbf{x}$$

- Note
- Remember the chain rule ->
- Followed by a vector derivative:
 - Second order moment (variance, assuming mean zero).
 - Assumes knowledge about probability density function!
- Example under assumption normally distributed data:

$$\blacksquare \frac{\partial}{\partial \mathbf{w}} (1 - \mathbf{w}^T \mathbf{x}) = \begin{bmatrix} \vdots \end{bmatrix} = -\mathbf{x}$$

- Thus:

$$\blacksquare \frac{\partial}{\partial \mathbf{w}} J(\mathbf{w}) = -\mathbb{E}[\mathbf{x}(y - \mathbf{w}^T \mathbf{x})] = \mathbf{0}$$

- Finally:

$$\blacksquare \widehat{\mathbf{w}}$$

Interactive: MSE decision boundary in 2D

- Pick a setting below to see how the MSE classifier partitions the two classes in 2D.
- Toggle the **decision boundary** to overlay the linear separator that the MSE solution (with targets $y \in \{0, 1\}$) places at $\mathbf{w}^T \mathbf{x} = 0.5$.

Least sum of error squares (LS) classifier

- Drawback of MSE classifier:
 - Assumes knowledge about the underlying distribution -> why not use Bayes classifier?
- How can we design a linear classifier without relying on knowledge about the distribution of the data?

Least sum of error squares cost function

- Consider the following loss function:
- $J(\mathbf{w}) = \sum_{i=1}^N (y - \mathbf{w}^T \mathbf{x}_i)^2$

Differentiating and setting to zero

- $\frac{\partial}{\partial \mathbf{w}} J(\mathbf{w}) = \mathbf{0} \rightarrow \sum_{i=1}^N$
 $-\mathbf{x}_i (y - \mathbf{w}^T \mathbf{x}_i) = 0$
- Collecting terms and rearranging:

Putting it all together

- Assume data is stored columnwise.
 - $\mathbf{X} =$
 - $\mathbf{y} =$
- Then, we get:
 - $(\mathbf{X}^T \mathbf{X}) \mathbf{w} = \mathbf{X}^T \mathbf{y}$
 - $\mathbf{w}$

Understanding the LS classifier

- Basically, MSE in practice. Optimal in the case of normal data.
- Connected to Bayes classifier under normal and equal variance assumption.
- Goes way back:
 - One of the earliest known uses is by Gauss for predicting motion of celestial bodies.
 - Astronomer Piazzi discovered Ceres. He made some observation before Ceres was obscured by the sun.
 - When would it return?
 - Could rely on equations for planetary motion.
 - Gauss used a least sum of error squared approach using the data collected by Piazzi.
- Least sum of error squared approaches are used all the time.

Limitations of MSE and LS classifier

- $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
- What could be potential problems?

Widow-Hoff algorithm (1960)

- Idea: iterative gradient descent on MSE loss.
- $\widehat{\mathbf{w}}(k) = \widehat{\mathbf{w}}(k-1)$

$$+ \rho_k \frac{\partial}{\partial \widehat{\mathbf{w}}(k-1)} J(\widehat{\mathbf{w}}(k-1))$$

Derivative of Widrow-Hoff

- Need to calculate

$$\frac{\partial}{\partial \widehat{\mathbf{w}}(k-1)} J$$
$$(\widehat{\mathbf{w}}(k-1))$$

Example of Widrow-Hoff update

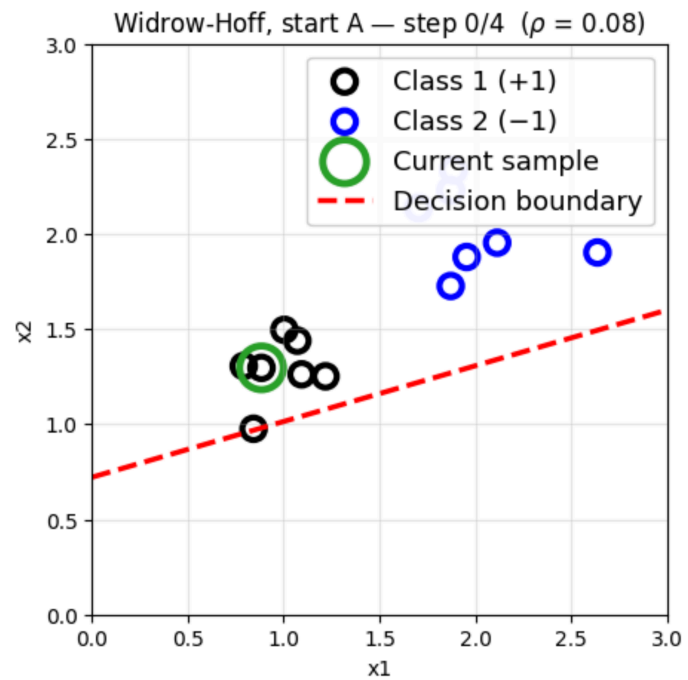
Interactive: Widrow-Hoff update

- Start from *random* initial weights and click **Next update** to step through four Widrow-Hoff gradient updates: $\mathbf{w}(k+1) = \mathbf{w}(k) + \rho(y - \mathbf{w}^T \mathbf{x})\mathbf{x}$.
- The point consumed by the most recent update is highlighted in red so you can see which sample drove the new boundary.

Step 0-4

- ☒ Step 0
- ☐ Step 1
- ☐ Step 2
- ☐ Step 3
- ☐ Step 4

Step 0 / 4 — weights: [0.152 -0.52 0.375]



Interactive: Widrow-Hoff update (different start)

- Same four updates, same data — only the random initialization changes. Compare the trajectory to the previous slide.

Step 0-4

☒ Step 0

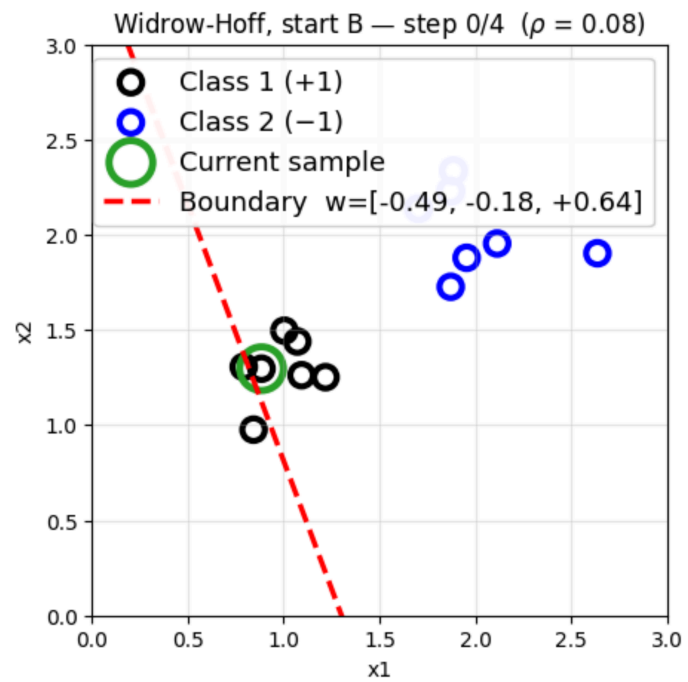
☐ Step 1

☐ Step 2

☐ Step 3

☐ Step 4

Step 0 / 4 — weights: $[-0.495 \ -0.184 \ 0.644]$



How do we stop?

- Check value of cost function

$$J(\widehat{\mathbf{w}}(k))$$

$$\simeq J(\widehat{\mathbf{w}}(k-1))$$

Testing

- $\hat{\mathbf{w}}$ is now fixed!

The Perceptron

- Proposed by Frank Rosenblatt in the 1950s
- A model of neurons in the brain.
- How to interpret the components in the linear classifiers we have seen so far?
 - $\mathbf{x}$: sensory input
 - $w_1, \dots, w_d$: synapses with different strenghts
 - f : a neuron fires or not.

The online/iterative Perceptron algorithm

- $\widehat{\mathbf{w}}(k) = \widehat{\mathbf{w}}(k-1) + \rho_k f(\mathbf{x}_k) \mathbf{x}_k$
- If $\text{sign}(y_k) \neq \text{sign}(f(\mathbf{x}_k))$
 - missclassified.
- Hence; do nothing if correctly classified and update if missclassified
- When do we stop?
- Can update in a batch-wise manner.

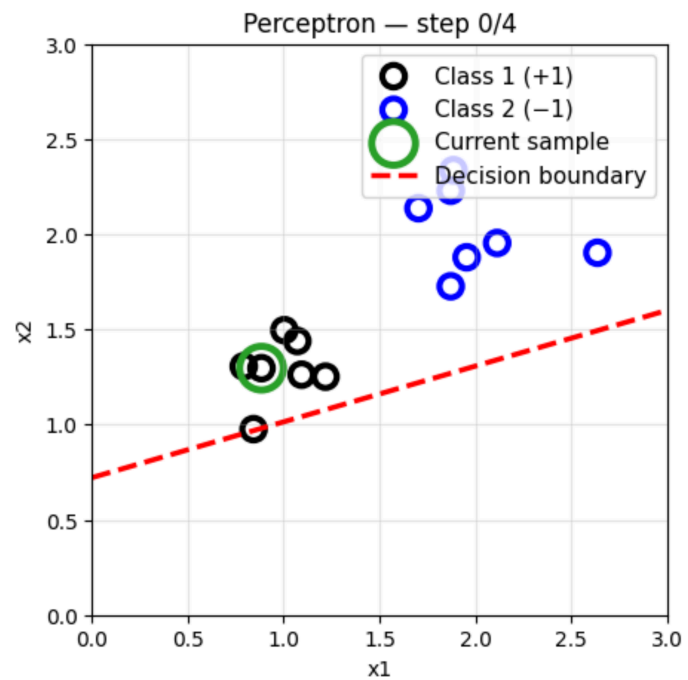
Example

- Same data and the same random initialization as the first Widrow-Hoff slide, but now we apply the **perceptron** rule: $\mathbf{w}(k+1) = \mathbf{w}(k) + \rho y_k \mathbf{x}_k$ only when $\text{sign}(\mathbf{w}^T \mathbf{x}_k) \neq y_k$.

Step (0..4)

- ☒ Step 0
- ☐ Step 1
- ☐ Step 2
- ☐ Step 3
- ☐ Step 4

Step 0 / 4 — weights: [0.152 -0.52 0.375]



Interactive: Perceptron update (different learning rate)

- Same four updates, same data — only the learning rate is different. Compare the trajectory to the previous slide.

Step (0..4)

- ☒ Step 0
- ☐ Step 1
- ☐ Step 2
- ☐ Step 3
- ☐ Step 4

Step 0 / 4 — weights: [0.152 -0.52 0.375]

